Thetawave.ai

💻 Ticket System 狀態模式設計與實作解析

Generated by Thetawave.ai

🎟️ 購票機系統（Ticket System）設計需求與分析

系統功能與行為規則

- **購票機用途**：讓學員以硬幣購買門票，每張票價為 3 枚硬幣。
- **硬幣投入限制**：一次只能投入 1 枚硬幣。
- **票機狀態分為兩種**：
  - **有票存（In stock）**：可接受硬幣，根據硬幣投入數量決定是否可購票。
  - **已售完（Sold out）**：無票存，投入硬幣時會直接退回硬幣。
- **硬幣投入後判斷**：
  - 若硬幣總數 ≥ 3（價格），購票燈亮起，表示可按購買鈕。
  - 燈亮時按購買鈕可購票並重置硬幣數。
  - 燈未亮時只能繼續投入硬幣。
- **退錢功能**：按退錢鈕時，退回所有已投入硬幣，並重置硬幣數與熄燈。
- **補充票存**：維護者可補充門票至票機。

物件導向分析（OOA）

類別與屬性

屬性名稱	說明	資料型態	備註
Tickets	庫存門票數量	整數	代表票機內剩餘票數
Total	硬幣總數	整數	票機內目前投入硬幣的總數
Light on	購買燈狀態	布林值（Boolean）	True 為亮燈，False 為熄燈
Price	門票價格	整數	靜態屬性，固定為 3

操作（方法）

方法名稱	說明	公開性
Insert coin	投入一枚硬幣，更新硬幣總數或退幣	公開
Press buy button	按購買鈕，若燈亮則購買門票	公開
Press refund button	按退幣鈕，退回所有硬幣並更新狀態	公開
Fill tickets	補充門票存量	公開
Update coins display	更新螢幕顯示的硬幣總數	私有
Turn light	控制購買燈開啟或熄滅	私有
Spit coins	退幣操作	私有
Issue one ticket	發出一張門票	私有

🛠️ 重要操作行為解析

Insert coin 行為流程

1. 投入一枚硬幣，硬幣總數加 1。
2. 判斷票存是否存在：
   - **有票存且硬幣總數 ≥ 價格**：

Thetawave.ai

    - 更新螢幕顯示。
    - 亮起購買燈，不再接受硬幣。
- **有票存但硬幣不足**：
    - 更新螢幕顯示，繼續接受硬幣。
- **無票存（售完）**：
    - 直接吐出剛投入的硬幣。

Press buy button 行為流程

1. 判斷購買燈是否亮起：
    - **亮起**：
        - 扣除價格硬幣數，硬幣總數歸零。
        - 更新螢幕顯示。
        - 發出一張門票。
        - 熄滅購買燈。
    - **未亮起**：
        - 不允許購買，印出錯誤訊息。

Press refund button 行為流程

1. 退回所有硬幣。
2. 更新螢幕顯示硬幣數為零。
3. 若購買燈亮起，則熄滅燈光。

🔁 狀態機（STATE MACHINE）觀點

狀態定義與轉移

購票機存在三個主要狀態：

狀態名稱	狀態說明
Sold out	無票存，投入硬幣皆退回
In stock	有票存，硬幣尚未足夠購買
Enough coins	有票存且硬幣已足夠購買，購買燈亮起

狀態轉移規則概述

目前狀態	觸發事件	條件(Guard)	行動(Action)	下一狀態
In stock	Insert coin	Total < Price	更新硬幣顯示(update coins display)	In stock
In stock	Insert coin	Total ≥ Price	更新硬幣顯示，亮燈(turn light on)	Enough coins
Enough coins	Press buy button	Tickets > 0	發票(issue one ticket)，更新硬幣顯示，熄燈(turn light off)	In stock
Enough coins	Press buy button	Tickets = 0	發票(issue one ticket)，更新硬幣顯示，熄燈	Sold out
Sold out	Insert coin	無條件	退回硬幣(spit coins)	Sold out
Sold out	Fill tickets	補充票存	進入 In stock 狀態	In stock

Thetawave.ai

狀態機圖的元素解析

- **狀態（State）**：以圓角矩形表示，內含狀態名稱與可選的 entry action、exit action。
- **狀態轉移（Transition）**：箭頭連接兩狀態，附帶三要素：
  - **觸發事件 (Trigger)**：事件名稱，如 Insert coin。
  - **成立條件 (Guard)**：以中括號表示，如 [Total ≥ Price]。
  - **行動 (Action)**：斜線後的動作，如 update coins display。

Entry 與 Exit Action 範例

- 進入 **Enough coins** 狀態時，執行亮燈（turn light on）。
- 離開 **Enough coins** 狀態時，執行熄燈（turn light off）。

🏷️ 類別圖擴充與屬性維護

新增狀態屬性與改寫行為判斷基準

- 新增屬性：**state**，用以代表 Ticket System 的當前狀態（In stock、Sold out、Enough coins）。
- 所有方法行為改為根據 **state** 來判斷動作，而非單純屬性判斷。

屬性與狀態一致性規則

狀態	屬性條件
Sold out	Tickets = 0，Total = 0
In stock	Tickets > 0，Total < Price
Enough coins	Tickets > 0，Total = Price

Enter state 方法行為流程

步驟	說明
1. 離開當前狀態	若為 Enough coins，熄滅購買燈
2. 更新狀態	將 state 設為新狀態
3. 進入新狀態	根據狀態調整屬性與執行 entry action：
	- **Enough coins**：Total 設為 Price，購買燈亮起
	- **In stock**：若 Total ≥ Price，退回多餘硬幣
	- **Sold out**：Total 退回所有硬幣，Tickets 設為 0

💻 程式碼實作重點說明

Insert coin 方法改寫

```
def insert_coin(self):
    if self.state == "Enough coins":
        print("Error: Cannot insert more coins, already enough.")
        return
    self.total += 1
    if self.state == "In stock":
        self.update_coins_display()
        if self.total >= self.price:
            self.enter_state("Enough coins")
    elif self.state == "Sold out":
```

Thetawave.ai

```
        self.spit_coins(1)
```

- 以狀態作為行為判斷基準。
- 投入硬幣後依狀態更新或退幣。

Press buy button 方法改寫

```
def press_buy_button(self):
    if self.state != "Enough coins":
        print("Error: Not enough coins to buy ticket.")
        return
    if self.tickets > 0:
        self.total -= self.price
        self.update_coins_display()
        self.issue_one_ticket()
        if self.tickets == 0:
            self.enter_state("Sold out")
        else:
            self.enter_state("In stock")
    else:
        print("Error: No tickets left.")
```

- 僅在 Enough coins 狀態允許購買。
- 購買後依票數決定狀態轉移。

Press refund button 方法改寫

```
def press_refund_button(self):
    self.spit_coins(self.total)
    self.total = 0
    self.update_coins_display()
    if self.light_on:
        self.turn_light(False)
    if self.tickets == 0:
        self.enter_state("Sold out")
    else:
        self.enter_state("In stock")
```

- 退回所有硬幣並更新螢幕與燈光。
- 根據票存決定狀態。

🔍 測試案例解析

測試案例	操作流程描述	預期結果
Case 1	投入 3 枚硬幣後嘗試投入第 4 枚硬幣	第 4 枚硬幣被拒絕，顯示錯誤訊息
Case 2	投入 3 枚硬幣後按退錢鈕	退回 3 枚硬幣，購買燈熄滅
Case 3	投入 3 枚硬幣後按購買鈕	出票，硬幣數歸零，購買燈熄滅
Case 4	無硬幣狀態按購買鈕	顯示錯誤訊息，不允許購買
Case 5	無硬幣狀態按退錢鈕	退回 0 枚硬幣，狀態正常
Case 6	將最後一張票買完，票機售完狀態下投入硬幣	硬幣被立刻退回，燈不亮

Thetawave.ai

🔧 狀態機圖（State Machine Diagram）的實務應用

- 狀態機圖可清楚呈現複雜行為的狀態轉移和條件。
- 利用狀態機，開發者與測試工程師能更有效率地理解及測試系統行為。
- 直接切換狀態的設計，減少測試前置程序，提升測試效率。

💡 狀態模式（State Pattern）核心觀念

- 利用物件的**狀態（State）**來決定其行為，而非用大量條件判斷。
- 狀態間的轉移清晰，行為分散到各狀態，降低系統複雜度。
- 強調狀態與屬性的一致性維護，確保系統狀態準確反映內部資料。

狀態模式透過將不同狀態的行為封裝，達成行為分離與狀態管理的清晰結構，有助於系統維護與擴展。狀態轉移的明確規範使得行為流程更具可預測性，減少錯誤發生的可能，提升整體系統穩定性。

🎟️ Ticket System 的狀態模式設計

> **狀態模式（State Pattern）**：讓物件的行為隨著其內部狀態改變而改變，且讓狀態的行為封裝在不同狀態類別中，方便管理與擴充。

問題背景與需求

問題點（Problem）	說明
行為會隨狀態改變	Ticket system 在不同狀態下，行為不同（如投幣、退幣、購票等）。
大量條件判斷式	原本程式中，公開操作包含大量 if-else 判斷目前狀態，導致程式難讀且難維護。
允許外部直接切換狀態	客戶端希望能直接切換 Ticket system 的狀態，方便測試或回復狀態。

主要設計問題

- **如何設計 Ticket system 類別，使工程師能專注維護每個狀態的行為？**
- **如何讓 Ticket system 物件隨狀態改變，能抽換自身行為？**

狀態模式的解決方案

1. 封裝變動之處（Encapsulate what varies）

- **三個狀態類別**：
  - Instock（有票存）
  - Sold out（售罄）

Thetawave.ai

    - Enough Coins（硬幣足夠）
- 每個狀態類別封裝該狀態下特有的行為與狀態轉換操作（如按鈕事件）。
- 將狀態轉移事件定義為狀態類別的方法，例如 PressRefundButton、PressBuyButton。

2. 萃取共同行為（Abstract Common Behaviors）

- 建立一個抽象的 State 類別：
    - **抽象操作（Abstract Operations）**：InsertCoin、PressBuyButton（因不同狀態行為差異大）
    - **具體操作（Concrete Operations）**：EnterState、ExitState、PressRefundButton、FillTickets（行為較一致）
- 各子類別可覆寫抽象方法，實現各自的行為變異。
- 例如，Instock 狀態覆寫 InsertCoin 和 PressBuyButton，且標記 PressBuyButton 為不支援操作（Unsupported），若呼叫會印錯誤訊息。

3. 委派（Delegation）

- **Ticket system 作為 Context**，持有當前的 State 物件。
- 將所有會因狀態而變的公開操作交由當前 State 物件處理（委派）。
- State 物件可反向呼叫 Ticket system 的內部私有操作，如更新硬幣顯示、開關燈、吐幣、發票等。
- 私有操作權限設為 Package 權限，確保只有同一 Package 內的 State 類別可存取。

狀態生命周期管理

- 在 EnterState 方法中：
    1. 呼叫舊狀態的 ExitState 進行清理
    2. 更新當前狀態物件
    3. 呼叫新狀態的 EnterState 執行初始化動作
- 例如 Enough Coins 狀態：
    - 進入時會開燈（TurnLight(true)）
    - 離開時會關燈（TurnLight(false)）

各狀態類別實作重點

狀態類別	主要行為與規則
Sold out	- 投幣後立即吐回硬幣 - 按購買按鈕會出錯（無票可賣） - 放入票券後轉移到 Instock 狀態 - 進入時確保票數為 0，硬幣吐出
Instock	- 可接受投幣 - 按購買按鈕不合法（標記為 Unsupported） - 票數不能為 0 - 投幣數量不能超過票價，超過會吐幣
Enough Coins	- 投幣不合法 - 硬幣總額等於票價 - 按購買按鈕可購票 - 進入時開燈，離開時關燈 - 票數不可為 0，否則例外

Thetawave.ai

狀態模式的類別結構圖

類別	說明
TicketSystem (Context)	持有 State 物件，對外公開操作，行為委派給當前 State，並提供私有操作給 State 存取。
State (抽象類別)	定義抽象操作和具體操作，作為所有狀態類別的父類別。
InstockState	封裝有票狀態下的行為。
SoldOutState	封裝售罄狀態下的行為。
EnoughCoinsState	封裝硬幣足夠狀態下的行為。

狀態模式套用流程總結

步驟	說明
1. 列出狀態	明確定義所有可能狀態，並以屬性標示。
2. 繪製狀態機圖	捕捉公開操作的狀態轉移行為，方便分析狀態間轉換。
3. 依賴反轉三步驟 (重構)	(a) 封裝變動行為成不同狀態類別 (b) 萃取共同行為成抽象狀態類別 (c) Context 委派行為給 State 物件。

狀態模式的設計成果與優點

成果	說明
Client 可直接切換狀態	透過注入不同 State 物件，改變 TicketSystem 行為，方便測試與狀態還原。
行為封裝在狀態子類別中	各狀態的行為邏輯獨立，程式碼清晰且維護性高。
符合開放封閉原則（OCP）	新增狀態只需新增對應 State 類別，無需修改 TicketSystem 類別。
減少大量條件判斷式	TicketSystem 中不再撰寫複雜 if-else 判斷，委派給狀態物件處理。

難點釐清與重要解釋

- **狀態模式與狀態機的關係**
  狀態模式是以物件導向方式實作狀態機的設計模式。狀態機圖（State Machine Diagram）為分析工具，狀態模式為程式碼層級的具體實作。
- **Entry Action 與 Exit Action**
  狀態生命週期中的「進入狀態」與「離開狀態」動作，封裝於 EnterState 與 ExitState 方法中，實現狀態轉換時的必要行為，如開燈關燈、清理數據等。
- **雙向關聯設計**
  TicketSystem 與 State 之間為雙向關聯，TicketSystem 持有當前 State，State 持有 TicketSystem 的參考，方便狀態操作時呼叫 Context 的內部方法。
- **抽象類別操作的分類**
  將變化大的行為設為抽象方法，強制子類別覆寫；行為穩定的則做為具體方法提供預設實作。

Thetawave.ai

重要示意表格 — 狀態與操作對應

狀態類別	InsertCoin	PressBuyButton	PressRefundButton	FillTickets	EnterState 行為
InstockState	允許，增加硬幣	不合法（Unsupported）	退硬幣	增加票數，保持狀態	清理多餘硬幣，票數不能為0
SoldOutState	投幣後吐回硬幣	不合法，印錯誤訊息	退硬幣（0枚）	放票後轉換至 Instock	確保票數為0，吐出所有硬幣
EnoughCoinsState	不允許	允許，完成購票	退硬幣	不變	開燈

狀態模式設計的物件導向技術應用

技術	說明
封裝 (Encapsulation)	將狀態行為封裝於不同類別，減少程式碼耦合。
繼承 (Inheritance)	抽象 State 類別定義行為模板，具體狀態類別繼承並覆寫。
多型 (Polymorphism)	TicketSystem 呼叫狀態物件的同一方法，行為依狀態物件而異。
依賴反轉 (Dependency Inversion)	TicketSystem 不直接實作狀態行為，而依賴抽象 State，利於擴充與維護。

狀態模式將行為依狀態分散封裝，透過物件導向的多型機制委派行為，能有效降低複雜度與條件判斷的膨脹，提升系統維護性與擴充性。行為變動性由狀態類別管理，而 Context 類別則保持簡潔，兩者協同實現動態行為切換的需求。